

# EXPLAIN / EXPLAIN ANALYZE Cheat Sheet

Quick reference for reading PostgreSQL execution plans: node types, cost fields, key signals.

## Basic Syntax

```
EXPLAIN SELECT ...; -- show plan only (no execution)
EXPLAIN ANALYZE SELECT ...; -- execute + actual times
EXPLAIN (ANALYZE, BUFFERS) SELECT ...; -- + buffer hit/miss info
EXPLAIN (ANALYZE, BUFFERS, VERBOSE) SELECT ...; -- + column lists
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...; -- JSON output (for tooling)
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF) SELECT ...; -- disable per-node timing (less overhead)
EXPLAIN (ANALYZE, SUMMARY) SELECT ...; -- + summary line
```

**GOTCHA:** `EXPLAIN ANALYZE` **actually runs the query**. Do NOT run ANALYZE on slow DELETE/UPDATE/INSERT without a ROLLBACK or on production. Use `BEGIN; EXPLAIN ANALYZE ...; ROLLBACK;` for mutations.

## Reading an Execution Plan

### Output Format

```
Node Type (cost=startup..total rows=est_rows width=est_bytes)
         (actual time=startup..total rows=actual_rows loops=N)
-> Child Node ...
```

### Cost Fields Explained

| Field                      | Meaning                                                          |
|----------------------------|------------------------------------------------------------------|
| cost=X..Y                  | X = startup cost (before first row), Y = total cost              |
| rows=N                     | Planner's <b>estimated</b> row count                             |
| width=N                    | Estimated average row width (bytes)                              |
| actual time=X..Y           | X = ms to first row, Y = ms for all rows                         |
| actual rows=N              | <b>Actual</b> rows returned                                      |
| loops=N                    | How many times this node was executed (multiply actual by loops) |
| Buffers: shared hit=N      | Pages read from shared_buffers (cache hit)                       |
| Buffers: shared read=N     | Pages read from disk (cache miss)                                |
| Buffers: temp read/written | Temp files used (spill to disk)                                  |

**GOTCHA:** When `loops > 1`, `actual time` is per-loop. Total = `actual time * loops`.

# All Node Types Reference

## Scan Nodes

| Node              | Description                                         | When Used                                     |
|-------------------|-----------------------------------------------------|-----------------------------------------------|
| Seq Scan          | Read entire table sequentially                      | Low selectivity, small table, no useful index |
| Index Scan        | Use index → fetch heap rows one by one              | High selectivity, random I/O acceptable       |
| Index Only Scan   | Index satisfies all columns needed                  | Covering index (INCLUDE or all cols in index) |
| Bitmap Index Scan | Build bitmap of matching TIDs from index            | Moderate selectivity (1–10%)                  |
| Bitmap Heap Scan  | Use bitmap to fetch heap pages in order             | After Bitmap Index Scan                       |
| CTE Scan          | Scan a materialized CTE                             | After CTE materialization                     |
| Function Scan     | Scan set-returning function (e.g., generate_series) | FROM function_call()                          |
| Values Scan       | Scan VALUES(...) list                               | INSERT...VALUES / SELECT * FROM VALUES        |
| Subquery Scan     | Wrap a subquery as a scan                           | Subqueries in FROM                            |
| Tid Scan          | Lookup by physical tuple ID                         | WHERE ctid = '(0,1)'                          |
| Sample Scan       | TABLESAMPLE (random sample)                         | TABLESAMPLE clause                            |

## Join Nodes

| Node        | Description                                   | When Used                                          |
|-------------|-----------------------------------------------|----------------------------------------------------|
| Nested Loop | For each outer row, scan inner                | Outer is small; inner has index; small result      |
| Hash Join   | Build hash table from inner; probe with outer | Medium-large joins, no index                       |
| Merge Join  | Both sides sorted; merge                      | Both sides already sorted / sorted by indexed cols |

## Aggregate Nodes

| Node          | Description                           |
|---------------|---------------------------------------|
| Aggregate     | GROUP BY or plain aggregate           |
| HashAggregate | GROUP BY using hash table (in memory) |

|                |                                |
|----------------|--------------------------------|
| GroupAggregate | GROUP BY over pre-sorted input |
| MixedAggregate | Partial aggregation            |
| WindowAgg      | Window function                |

**Sort / Limit Nodes**

| Node             | Description                                     |
|------------------|-------------------------------------------------|
| Sort             | Full sort (may write temp files if > work_mem)  |
| Incremental Sort | Sorts on top of partially-sorted input (PG 13+) |
| Limit            | Apply LIMIT/OFFSET                              |
| Unique           | Remove duplicates (after sort)                  |

**Set Operation Nodes**

| Node        | Description                 |
|-------------|-----------------------------|
| Append      | UNION ALL, partition scan   |
| MergeAppend | Sorted UNION ALL            |
| BitmapAnd   | AND of two bitmap scans     |
| BitmapOr    | OR of two bitmap scans      |
| HashSetOp   | INTERSECT / EXCEPT via hash |

**Modification Nodes**

| Node                     | Description                   |
|--------------------------|-------------------------------|
| Insert / Update / Delete | DML operations                |
| LockRows                 | SELECT FOR UPDATE / FOR SHARE |

**Parallelism Nodes**

| Node                      | Description                                           |
|---------------------------|-------------------------------------------------------|
| Gather                    | Collect rows from parallel workers into single stream |
| Gather Merge              | Collect pre-sorted rows from parallel workers         |
| Parallel Seq Scan         | Parallel sequential scan                              |
| Parallel Index Scan       | Parallel index scan                                   |
| Parallel Bitmap Heap Scan | Parallel bitmap heap scan                             |
| Parallel Append           | Parallel union/partition scan                         |

---

# Key Signals to Look For

## RED FLAGS — Problem Indicators

| Signal                                        | What It Means                                      | Fix                                                  |
|-----------------------------------------------|----------------------------------------------------|------------------------------------------------------|
| Seq Scan on large table with high rows filter | No useful index, or planner chose not to use one   | Add index; check random_page_cost setting            |
| rows=100 actual rows=10000 (estimate too low) | Stale statistics, non-uniform data                 | ANALYZE table; or increase default_statistics_target |
| rows=100000 actual rows=1 (estimate too high) | Correlated columns or bad stats                    | Create multi-column statistics: CREATE STATISTICS    |
| Nested Loop with large outer + large inner    | Cartesian explosion, missing index on inner        | Add index on inner join column                       |
| Sort with method: external                    | Work_mem exceeded, spill to disk                   | Increase work_mem for this query                     |
| Buffers: shared read=NNNN (high disk reads)   | Buffer cache misses, cold cache or index too large | Increase shared_buffers, check index bloat           |
| Buffers: temp=NNNN                            | Hash/sort spilled to disk                          | Increase work_mem                                    |
| Filter: (condition) AFTER scan                | Rows fetched then filtered (index could help)      | Add index for that condition                         |
| Rows Removed by Filter: NNNN                  | Large removal = wasteful scan                      | Better index or query rewrite                        |
| loops=N actual time per loop × N = huge       | Correlated subquery executing N times              | Rewrite as JOIN or window function                   |
| Hash Batches: > 1                             | Hash table spilled to disk                         | Increase work_mem                                    |
| Heap Fetches: NNNN on Index Only Scan         | Visibility map not clean, must check heap          | Run VACUUM table                                     |

## GREEN SIGNALS — Healthy Plan

| Signal                                   | Meaning                           |
|------------------------------------------|-----------------------------------|
| Index Only Scan                          | No heap access needed (best case) |
| Estimates close to actuals (within 2-3x) | Good statistics                   |
| Hash Join on large tables                | Efficient for large joins         |
| Merge Join when both sides sorted        | Very efficient join               |
| Parallel Seq Scan on huge table          | Parallelism active                |
| Gather Merge                             | Parallel + sorted                 |

|                                     |               |
|-------------------------------------|---------------|
| Low Buffers: shared read (all hits) | Data in cache |
|-------------------------------------|---------------|

## Worked Example: Reading a Full Plan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.country = 'US'
      AND o.created_at >= '2024-01-01'
GROUP BY c.name;
```

### Sample output:

```
HashAggregate  (cost=1234.56..1256.78 rows=1823 width=36)
               (actual time=45.123..46.001 rows=1800 loops=1)
  Group Key: c.name
  Batches: 1  Memory Usage: 512kB
  Buffers: shared hit=823 read=210
->  Hash Join  (cost=234.50..1198.33 rows=7247 width=28)
       (actual time=2.345..38.901 rows=7100 loops=1)
    Hash Cond: (o.customer_id = c.customer_id)
    Buffers: shared hit=723 read=210
->   Index Scan using idx_orders_date on orders o
        (cost=0.56..890.12 rows=9823 width=16)
        (actual time=0.056..22.345 rows=9700 loops=1)
      Index Cond: (created_at >= '2024-01-01')
      Buffers: shared hit=456 read=210
->   Hash      (cost=178.00..178.00 rows=4520 width=20)
        (actual time=1.234..1.234 rows=4489 loops=1)
      Buckets: 4096  Batches: 1  Memory: 384kB
      Buffers: shared hit=267
->    Seq Scan on customers c
        (cost=0.00..178.00 rows=4520 width=20)
        (actual time=0.012..0.987 rows=4489 loops=1)
      Filter: (country = 'US')
      Rows Removed by Filter: 6911
      Buffers: shared hit=267

Planning Time: 0.456 ms
Execution Time: 46.234 ms
```

### Analysis:

- Orders: Index Scan on date — good. 9700 actual rows, 9823 estimated — excellent estimates.
- Buffers: shared read=210 — 210 pages from disk. If this is repeated: consider caching.
- Customers: Seq Scan + Filter — Removed 6911/11400 rows. Could add `CREATE INDEX ON customers(country)` if `country='US'` is selective.
- HashAggregate in memory (Batches: 1) — no spill. Good.
- Total: 46ms. Acceptable for this query.

## Useful EXPLAIN Queries

```
-- Enable pg_stat_statements first
CREATE EXTENSION pg_stat_statements;

-- Find slowest queries
SELECT
    substring(query, 1, 100) AS query,
    calls,
    ROUND(total_exec_time::numeric / calls, 2) AS avg_ms,
    ROUND(total_exec_time::numeric, 0) AS total_ms,
    rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;

-- Queries with worst estimation errors (planner confusion)
SELECT
    substring(query, 1, 100),
    calls,
    rows / calls AS avg_actual_rows
FROM pg_stat_statements
WHERE calls > 100
ORDER BY rows / calls DESC
LIMIT 20;
```

## auto\_explain — Log Slow Query Plans Automatically

```
# postgresql.conf
shared_preload_libraries = 'auto_explain'
auto_explain.log_min_duration = 1000    # Log plans for queries > 1 second
auto_explain.log_analyze = on
auto_explain.log_buffers = on
auto_explain.log_format = text          # or json
auto_explain.log_nested_statements = on
```

## Common Optimization Actions from EXPLAIN Output

| What EXPLAIN Shows               | Action                                                            |
|----------------------------------|-------------------------------------------------------------------|
| Seq Scan on large table          | CREATE INDEX ON table (filtered_column)                           |
| Seq Scan with Filter             | CREATE PARTIAL INDEX ... WHERE condition                          |
| Poor row estimates               | ANALYZE table; Or ALTER TABLE t ALTER COLUMN c SET STATISTICS 500 |
| Correlated multi-column estimate | CREATE STATISTICS stats_name ON col1, col2 FROM table             |

|                                |                                                                        |
|--------------------------------|------------------------------------------------------------------------|
| Nested Loop on large sets      | Check missing index on inner table join column                         |
| Sort with external sort        | SET work_mem = '256MB'; (session level)                                |
| Hash Batches > 1               | Increase work_mem                                                      |
| Index Only Scan + Heap Fetches | VACUUM table to clean visibility map                                   |
| Wrong join order               | SET join_collapse_limit = 1; then manually reorder                     |
| Parallel not used              | Check max_parallel_workers_per_gather,<br>min_parallel_table_scan_size |

## Cost Model Reference

| Setting              | Default | What It Controls                           |
|----------------------|---------|--------------------------------------------|
| seq_page_cost        | 1.0     | Cost of reading one sequential page        |
| random_page_cost     | 4.0     | Cost of random page read (set 1.1 for SSD) |
| cpu_tuple_cost       | 0.01    | Cost to process one row                    |
| cpu_index_tuple_cost | 0.005   | Cost to process one index entry            |
| cpu_operator_cost    | 0.0025  | Cost of applying one operator              |
| parallel_tuple_cost  | 0.1     | Extra cost of passing tuple to Gather      |
| parallel_setup_cost  | 1000    | Fixed cost to start parallel workers       |

```
-- Force index use (diagnostic only – don't use in production permanently)
SET enable_seqscan = OFF;
EXPLAIN SELECT ...;
SET enable_seqscan = ON;

-- Force specific join type
SET enable_nestloop = OFF;
SET enable_hashjoin = OFF;
-- These are diagnostic tools only!
```